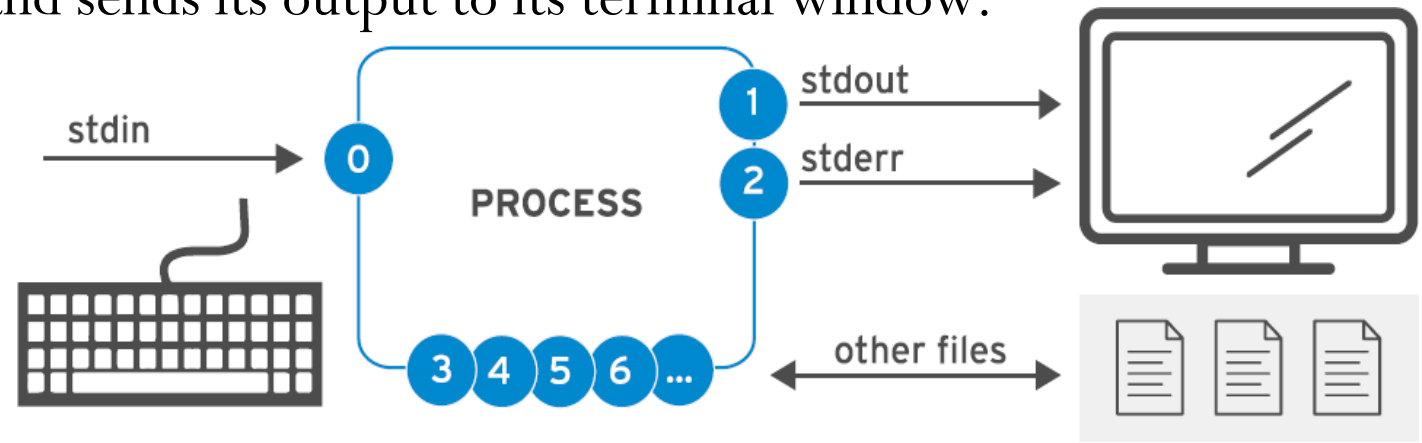
Redirection and pipeline in Linux

Outile

- Redirection
- Pipeline

STANDARD INPUT, STANDARD OUTPUT, AND STANDARD ERROR

- A running program, or *process*, needs to read input from somewhere and write output to somewhere. A command run from the shell prompt normally reads its input from the keyboard and sends its output to its terminal window.

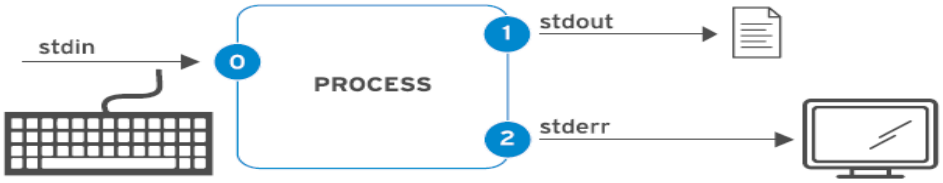
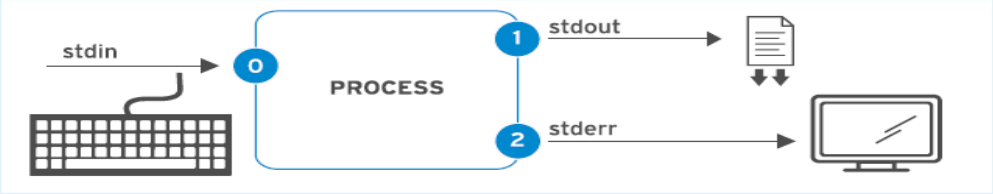
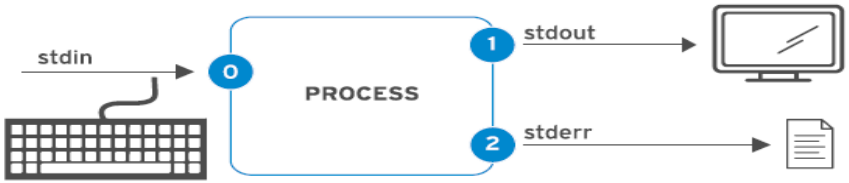


: Process I/O channels (file descriptors)

NUMBER	CHANNEL NAME	DESCRIPTION	DEFAULT CONNECTION	USAGE
0	**stdin**	Standard input	Keyboard	read only
1	**stdout**	Standard output	Terminal	write only
2	**stderr**	Standard error	Terminal	write only
3+	***filename***	Other files	none	read and/or write

REDIRECTING OUTPUT TO A FILE

Output Redirection Operators

USAGE	EXPLANATION	VISUAL AID
> file	redirect **stdout** to overwrite a file	
>> file	redirect **stdout** to append to a file	
2> file	redirect **stderr** to overwrite a file	
2> /dev/null	discard **stderr** error messages by redirecting to **/dev/null**	

> *file* 2>&1 &> *file*	redirect **stdout** and **stderr** to overwrite the same file	
>> *file* 2>&1 &>> *file*	redirect **stdout** and **stderr** to append to the same file	

Examples for Output Redirection

- Save a time stamp for later reference.

```
[user@host ~]$ date > /tmp/saved-timestamp
```

- Copy the last 100 lines from a log file to another file.

```
[user@host ~]$ tail -n 100 /var/log/dmesg > /tmp/last-100-boot-messages
```

- Concatenate four files into one.

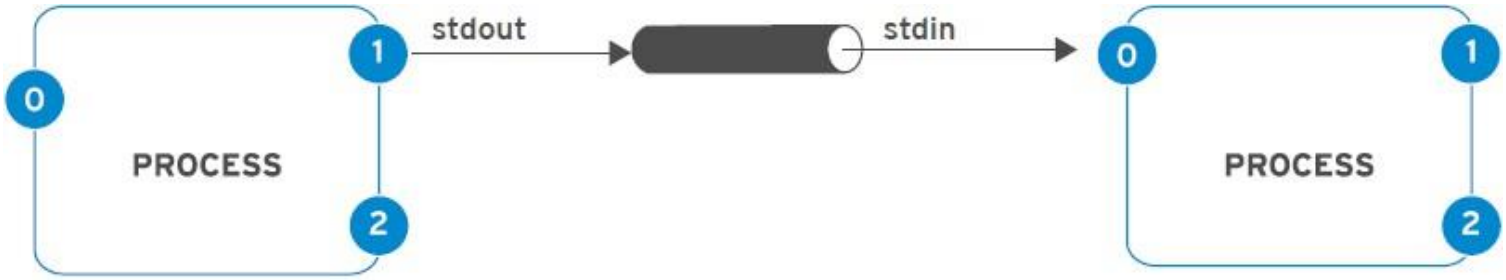
```
[user@host ~]$ cat file1 file2 file3 file4 > /tmp/all-four-in-one
```

- List the home directory's hidden and regular file names into a file.

```
[user@host ~]$ ls -a > /tmp/my-file-names
```

CONSTRUCTING PIPELINES

- A *pipeline* is a sequence of one or more commands separated by the *pipe* character (|).
- A pipe connects the standard output of the first command to the standard input of the next command.



Process I/O piping

Pipeline Examples

This example takes the output of the **ls** command and uses **less** to display it on the terminal one screen at a time.

```
[user@host ~]$ ls -l /usr/bin | less
```

The output of the **ls** command is piped to **wc -l**, which counts the number of lines received from **ls** and prints that to the terminal.

```
[user@host ~]$ ls | wc -l
```

In this pipeline, **head** will output the first 10 lines of output from **ls -t**, with the final result redirected to a file.

```
[user@host ~]$ ls -t | head -n 10 > /tmp/ten-last-changed-files
```

Pipelines, Redirection, and the tee Command

- **tee** copies its standard input to its standard output and also redirects its standard output to the files named as arguments to the command.

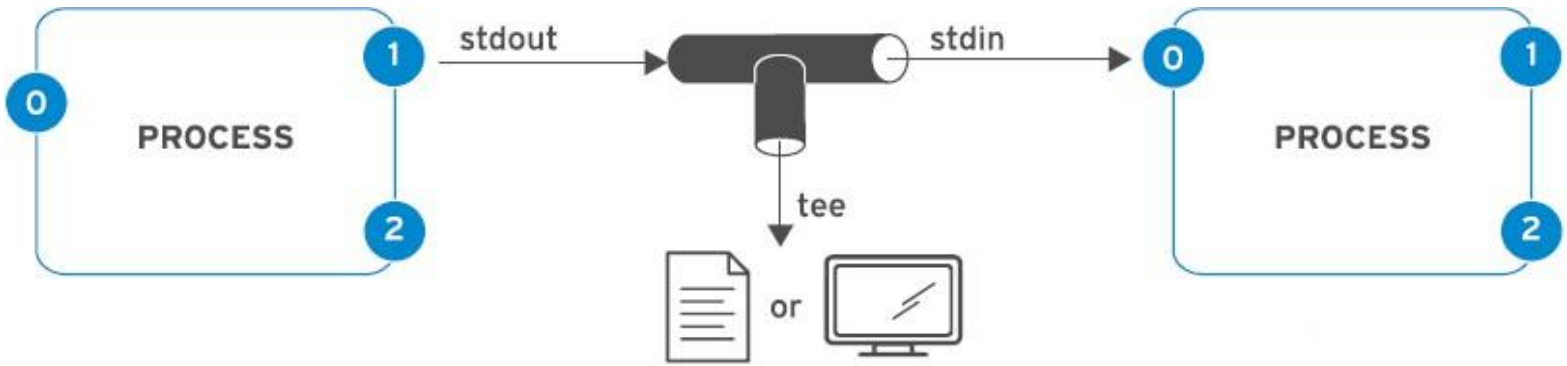


Figure 5.9: Process I/O piping with tee

Pipeline Examples Using the tee Command

This example redirects the output of the **ls** command to the file and passes it to **less** to be displayed on the terminal one screen at a time.

```
[user@host ~]$ ls -l | tee /tmp/saved-output | less
```

If **tee** is used at the end of a pipeline, then the final output of a command can be saved and output to the terminal at the same time.

```
[user@host ~]$ ls -t | head -n 10 | tee /tmp/ten-last-changed-files
```